

Exercise 7: Deploying a Deep Learning Model to Azure ML

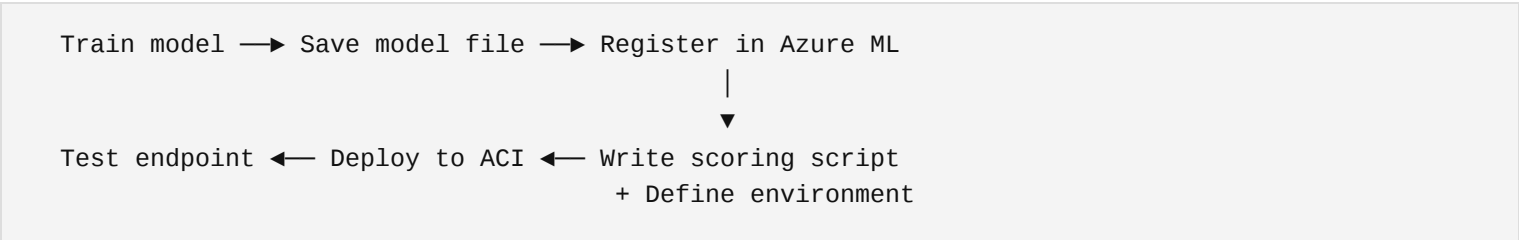
Module 2: Deep Learning and Neural Networks

Duration: 60 minutes | Platform: Google Colab + Azure ML

What you will build: A trained Keras model deployed as a live REST API on Azure Container Instance. By the end, you will be able to send a JSON request and receive a real-time prediction in response.

1. Conceptual Overview

Training a model is only half the job. For a model to be useful in a product it must be reachable by other systems — a mobile app, a dashboard, or a downstream data pipeline. Azure Machine Learning handles this through a standardised deployment pipeline:



Stage	What happens	Output
Train	Model learns from data	<code>.h5</code> model file
Register	File versioned and stored in workspace blob storage	Model object with name + version
Scoring script	Python file that loads model and handles one prediction request	<code>score.py</code>
Environment	Specifies Python packages needed at inference time	Conda/pip spec
Deploy	Azure packages everything into a Docker container and exposes a URL	Live REST endpoint
Test	Send JSON to the URL, receive predictions	JSON response

2. Prerequisites

- Azure account with an active subscription (student free credit is sufficient)
- Azure ML Workspace already created: `ml-workshop-ws` in `ml-workshop-rg`
- `config.json` downloaded from Azure ML Studio and uploaded to Colab

How to download config.json

1. Go to ml.azure.com and open your workspace
2. Click the workspace name in the top-right corner

3. Select **Download config file**

4. Upload the file to your Colab session using the left-panel file browser

Never commit `config.json` to a public GitHub repository. It contains your subscription ID and workspace credentials.

3. Step-by-Step Guide

1 Install the Azure ML SDK

```
!pip install azureml-core azureml-sdk --quiet
```

Takes 2-4 minutes. The SDK lets you interact with Azure ML from any environment, including Colab.

2 Connect to Your Workspace

```
from azureml.core import Workspace
ws = Workspace.from_config()
print(f"Connected: {ws.name} in {ws.resource_group}")
```

Reads workspace details from `config.json`. A browser login prompt will appear on first use. Pass `path="./config.json"` explicitly if the file is not in the working directory.

3 Train and Save the Model

```
import tensorflow as tf, numpy as np, os

model = tf.keras.Sequential([
    tf.keras.Input(shape=(13,)),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(X_train, y_train, epochs=20, validation_split=0.2)

os.makedirs("outputs", exist_ok=True)
model.save("outputs/heart_model.h5")
```

Write to `outputs/` — Azure ML automatically captures anything in this folder as a run artifact.

4 Register the Model

```

from azureml.core.model import Model

registered_model = Model.register(
    workspace    = ws,
    model_path   = "outputs/heart_model.h5",
    model_name   = "heart-disease-classifier",
    description  = "Binary classifier: heart disease from 13 clinical features",
    tags         = {"framework": "keras", "task": "binary-classification"}
)
print(f"Registered: {registered_model.name} v{registered_model.version}")

```

Each re-registration of the same name auto-increments the version. Version history enables rollback.

5 Write the Scoring Script

```

%%writefile score.py
import json, numpy as np, tensorflow as tf
from azureml.core.model import Model

def init():
    global model
    model_path = Model.get_model_path("heart-disease-classifier")
    model = tf.keras.models.load_model(model_path)

def run(raw_data):
    try:
        features    = np.array(json.loads(raw_data)["data"], dtype=np.float32)
        predictions = model.predict(features).flatten().tolist()
        return json.dumps({"predictions": predictions})
    except Exception as e:
        return json.dumps({"error": str(e)})

```

`init()` runs once at container startup and loads the model into memory. `run()` runs on every prediction request.

Critical: Load the model in `init()`, not in `run()`. Loading in `run()` reloads from disk on every call, adding 3-8 seconds of latency per request.

6 Define the Inference Environment

```

from azureml.core import Environment
from azureml.core.conda_dependencies import CondaDependencies

env = Environment("heart-disease-inference-env")
deps = CondaDependencies()
deps.add_pip_package("tensorflow==2.10.0")
deps.add_pip_package("numpy==1.23.5")

```

```
deps.add_pip_package("azureml-defaults")    # required – do not remove
env.python.conda_dependencies = deps
```

Only include packages needed to run `score.py`. Avoid training-only packages (matplotlib, pandas) — they bloat the Docker image and increase deployment time.

7 Create Inference Configuration

```
from azureml.core.model import InferenceConfig

inference_config = InferenceConfig(entry_script="score.py", environment=env)
```

Bundles the scoring script and environment into a single deployable object.

8 Deploy to ACI

```
from azureml.core.webservice import AciWebservice

aci_config = AciWebservice.deploy_configuration(cpu_cores=1, memory_gb=1)

service = Model.deploy(
    workspace      = ws,
    name           = "heart-disease-api",
    models         = [registered_model],
    inference_config = inference_config,
    deployment_config = aci_config,
    overwrite      = True
)
service.wait_for_deployment(show_output=True)
print("State:", service.state)
print("URL  :", service.scoring_uri)
```

Deployment takes **5-10 minutes**. `show_output=True` streams logs in real time. If `service.state` is not `Healthy`, call `service.get_logs()` immediately.

9 Test the Endpoint

```
import requests, json

payload = json.dumps({"data": [[63,1,3,145,233,1,0,150,0,2.3,0,0,1]]})
headers = {"Content-Type": "application/json"}
response = requests.post(service.scoring_uri, data=payload, headers=headers)
```

```
print(response.status_code)    # 200 = success
print(response.json())         # {"predictions": [0.73]}
```

A successful call returns HTTP `200` with a JSON body. Values are sigmoid probabilities; ≥ 0.5 means the model predicts heart disease.

10 Clean Up

```
service.delete()
print("Service deleted")
```

Always run this before closing. A running ACI endpoint charges by the hour regardless of whether it receives any requests.

4. Deployment Checklist

- [] `config.json` uploaded and workspace connection successful
- [] Model saved to `outputs/` before registration
- [] `score.py` has both `init()` and `run()`
- [] Model loaded inside `init()`, not `run()`
- [] `azureml-defaults` included in pip packages
- [] `service.state == "Healthy"` before testing
- [] Test request returns HTTP 200 with correct predictions
- [] `service.delete()` called after exercise

5. Troubleshooting

Symptom	Likely Cause	Fix
<code>AuthenticationException</code>	Config not found or expired login token	Re-upload <code>config.json</code> ; re-run <code>Workspace.from_config()</code> and complete browser login
Deployment fails after 10 min	Error inside scoring script or missing package	Run <code>print(service.get_logs())</code> and read the Python traceback
<code>ModuleNotFoundError</code> in container logs	Package missing from environment definition	Add to <code>CondaDependencies</code> and redeploy

HTTP 500 from endpoint	Runtime error inside <code>run()</code>	Add try/except in <code>run()</code> returning <code>str(e)</code> ; check logs
HTTP 400: bad input	JSON structure does not match what <code>run()</code> expects	Print <code>raw_data</code> inside <code>run()</code> to inspect what arrives
<code>service.state == "Failed"</code>	Docker image build failed	Run <code>service.get_logs()</code> ; often caused by incompatible package versions

6. ACI vs AKS: When to Use Each

	Azure Container Instance (ACI)	Azure Kubernetes Service (AKS)
Use case	Testing, dev, low traffic	Production, high traffic, SLA requirements
Scaling	Manual, single container	Auto-scaling across multiple nodes
Cost	Low (pay per container-hour)	Higher (cluster always running)
Deployment time	5-10 minutes	10-20 minutes
This exercise	Yes	No (out of scope)

7. References and Learning Resources

Official Microsoft Documentation

Deploy models with Azure ML — Microsoft Docs

Official guide covering all deployment targets (ACI, AKS, local). The ACI section maps directly to this exercise.

Deploying models trained outside Azure ML

Exactly the workflow used here — registering a model trained locally and deploying it via the Python SDK.

Azure ML Environments — Concept Guide

Explains curated vs custom environments, pinning package versions, and how environments map to Docker images.

Microsoft Learn — End-to-End Deployment Tutorial

Interactive step-by-step tutorial with screenshots from workspace setup through endpoint testing. Recommended before running the notebook for the first time.

Azure ML Core Python SDK — API Reference

Full reference for all classes used in this exercise: `Workspace` , `Model` , `InferenceConfig` , `AciWebService` .

Video Tutorials

[Azure Machine Learning in 1 Hour — Microsoft Developer \(YouTube\)](#)

Full pipeline walkthrough. Skip to **38:00** for the deployment section directly relevant to this exercise.

[Deploying ML Models with Azure ML Studio — Pragmatic AI Labs \(YouTube\)](#)

Focuses on inference configuration, scoring script structure, and ACI deployment. Closest match to the exact workflow in this exercise.

[Azure ML Python SDK Tutorial — Krish Naik \(YouTube\)](#)

Full pipeline from data to deployed endpoint. Good for understanding each SDK call before running the notebook.

[Microsoft Learn Path — Azure ML Pipelines \(Free\)](#)

Structured learning path with sandbox exercises. Module 4 covers deployment and endpoint management interactively.

For Production Deployments (Further Reading)

[Monitoring with Application Insights](#)

Adding logging and telemetry to a deployed endpoint — the logical next step after this exercise.

[ACI vs AKS Deep Dive](#)

Detailed comparison of both deployment targets with guidance on when to switch from ACI to AKS in a real project.